Alex Aravena

Jefe de Proyecto y Apoderado · LafroX SpA

LAFROX-Formulario-T-10.pdf

Índice general



Metodología de desarrollo de software

La metodología de desarrollo se coordina con la gestión del proyecto y con sus entregas incrementales. Se utiliza **Rational Unified Process (RUP)**, un proceso iterativo e incremental que organiza el desarrollo en cuatro fases y permite gestionar requisitos, arquitectura, implementación y pruebas de manera progresiva. Su elección se basa en que el proyecto tiene requisitos regulatorios y de continuidad que exigen una arquitectura validada temprano, documentación trazable y entregas formales por etapa, sin renunciar a iteraciones cortas que reduzcan el tiempo hasta disponer de software verificable.

RUP se organiza en las siguientes fases:

1. **Inicio**: se delimitan el alcance y los objetivos de la solución, se identifican los principales interesados y requisitos y se elaboran las estimaciones iniciales. También se reconocen los riesgos principales y se establece una visión inicial del producto.
2. **Elaboración**: se profundizan los requisitos, incluidos los casos de uso, y se define la arquitectura de la solución. Se analizan los riesgos prioritarios y se prepara el plan de desarrollo, de modo que las decisiones de diseño más relevantes se validen antes de construir. Los prototipos con usuarios terminan antes de construir cada módulo.
3. **Construcción**: se desarrollan las capacidades priorizadas mediante iteraciones. En cada iteración se analizan los requisitos correspondientes, se diseña e implementa la solución y se realizan pruebas. Los resultados se revisan con los interesados pertinentes y se ajustan cuando corresponde.
4. **Transición**: se prepara la solución para su uso: pruebas de aceptación, resolución de observaciones, preparación de usuarios, marcha blanca y despliegue conforme a los hitos aprobados.

Las iteraciones producen resultados verificables, pero no implican una puesta en producción o una marcha blanca. La marcha blanca y la aceptación formal se realizan en los momentos definidos para cada etapa contractual. Los casos de uso expresan y siguen los requisitos funcionales, y la matriz del Formulario T-12 los vincula con su prueba.

La secuencia de iteraciones se coordina con las dos etapas contractuales: en la Etapa 1, preventa, recepción, bodega, preparación y cross-docking; trazabilidad de lotes y de la cadena de frío; planificación de rutas; confirmación de recepción por los conductores; y manejo de efectivo y retiros. En la Etapa 2, los portales y el intercambio electrónico con los clientes y, luego, el costo de servir. La transición sigue el cronograma: marcha blanca de la Etapa 1 entre los meses 13 y 15 y paso a producción en el mes 16; marcha blanca de la Etapa 2 en los meses 19 y 20 y paso a producción en el mes 21.



1.0.1 Arquitectura evolutiva, refactorización y deuda técnica

La arquitectura se establece durante Elaboración y se aprueba en el H2; se valida progresivamente en las iteraciones de Construcción. Cuando los nuevos requisitos o los resultados de las pruebas justifican cambios, la arquitectura se ajusta mediante una nueva versión de la decisión en el registro de decisiones de arquitectura del SD4, con su fecha, estado y acta del Comité de Arquitectura.

Durante la construcción, el equipo refactoriza el software para mejorar su estructura interna y facilitar su mantenimiento, preservando su comportamiento funcional; las pruebas automatizadas comprueban que el comportamiento no cambió. Las limitaciones o compromisos técnicos que puedan dificultar cambios futuros se registran como deuda técnica, con su impacto y prioridad. Una deuda clasificada como bloqueante por el análisis estático impide el despliegue; las demás se planifican en las iteraciones y se revisan en el Comité de Arquitectura.

1.0.2 DevSecOps, integración y entrega continuas, infraestructura como código y pruebas automatizadas

Cada cambio pasa por el pipeline de integración continua que define el SD4, sección 4.2: GitLab CI orquesta y AWS CodeBuild construye cada imagen de forma hermética, con procedencia SLSA nivel 3. El pipeline ejecuta la auditoría de dependencias con `composer audit`, las pruebas PHPUnit, el análisis estático con PHPStan y Larastan, el formato con Laravel Pint, las pruebas de contrato contra OpenAPI 3.1 y AsyncAPI 2.6, el escaneo de secretos y de imágenes de contenedor y la medición de cobertura.

Las compuertas son bloqueantes, no opcionales. El pipeline detiene la promoción ante:

- un hallazgo de seguridad crítico o alto en dependencias, código, secretos o imagen;
- un contrato público roto sin una nueva edición de la interfaz;
- una cobertura de la lógica de negocio inferior al 70 % (RT-04.11);
- una cobertura de líneas por pruebas unitarias del código modificado inferior al 80 %, política corporativa de LafroX (paquete 1.5.2);
- una deuda técnica bloqueante o una prueba en falla.

La imagen aprobada se firma, se publica en Elastic Container Registry y se promueve por su digest, de modo que ningún ambiente recompila. La infraestructura de los cinco ambientes se describe como código: Terraform administra los recursos con estados separados por ambiente y Ansible configura los hosts; cada recurso tiene un único propietario de código. Las migraciones de base de datos siguen la estrategia de expandir y contraer y declaran su reversión.

La entrega continua mantiene versiones verificadas y listas para su despliegue en cada ambiente. El paso a Producción se ejecuta sólo dentro de las ventanas permitidas por el caso y, durante el desarrollo, sólo en los hitos aprobados: ningún cambio se despliega en septiembre, en diciembre ni en los tres primeros días hábiles de un mes.



Media firma: Alex Aravena

1.0.3 Ceremonias, cadencias y decisiones del desarrollo

El trabajo del equipo de desarrollo se organiza en iteraciones de Construcción de dos semanas. Al inicio de cada iteración se revisan los requisitos y prioridades, se seleccionan las tareas que caben en la capacidad registrada y se aclaran sus criterios de aceptación. Durante la iteración, el equipo sigue el avance en el tablero Kanban, con bloqueos, responsables y dependencias.

Al cierre de cada iteración se demuestra el incremento verificable y se contrasta con sus criterios de aceptación; cuando es pertinente, se presenta a los interesados para recoger observaciones. El equipo revisa además las dificultades y aprendizajes de la iteración y registra las acciones de mejora con su responsable. Ninguna demostración constituye por sí misma una entrega formal o una puesta en producción.

Los artefactos del desarrollo son los casos de uso, la matriz de trazabilidad del Formulario T-12, el registro de decisiones de arquitectura, los contratos de interfaz, el registro de deuda técnica y los informes de pruebas de cada iteración.

Las decisiones técnicas necesarias para implementar los requisitos se toman dentro del equipo responsable y se documentan cuando afectan la arquitectura, las integraciones, la seguridad o la operación; las que cambian la arquitectura pasan por el Comité de Arquitectura. Si una decisión modifica el alcance, el plazo, los costos o los criterios de aceptación aprobados, se gestiona como una solicitud de cambio según la sección 6.1.4.

